4.1 Illustrative Examples

4.1.1 The Factorial Function

To demonstrate the mechanics of recursion, we begin with a simple mathematical example of computing the value of the ***factorial function***. The factorial of a positive integer n, denoted $n!$, is defined as the product of the integers from 1 to n. If $n = 0$, then $n!$ is defined as 1 by convention. More formally, for any integer $n \geq 0$,

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1 & \text{if } n \geq 1. \end{cases}$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$. The factorial function is important because it is known to equal the number of ways in which n distinct items can be arranged into a sequence, that is, the number of ***permutations*** of n items. For example, the three characters `a`, `b`, and `c` can be arranged in $3! = 3 \cdot 2 \cdot 1 = 6$ ways: `abc`, `acb`, `bac`, `bca`, `cab`, and `cba`.

There is a natural recursive definition for the factorial function. To see this, observe that $5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) = 5 \cdot 4!$. More generally, for a positive integer n, we can define $n!$ to be $n \cdot (n-1)!$. This ***recursive definition*** can be formalized as

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1. \end{cases}$$

This definition is typical of many recursive definitions. First, it contains one or more ***base cases***, which are defined nonrecursively in terms of fixed quantities. In this case, $n = 0$ is the base case. It also contains one or more ***recursive cases***, which are defined by appealing to the definition of the function being defined.

A Recursive Implementation of the Factorial Function

Recursion is not just a mathematical notation; we can use recursion to design a Python implementation of a factorial function, as shown in Code Fragment 4.1.

```
1 def factorial(n):
2   if n == 0:
3     return 1
4   else:
5     return n * factorial(n-1)
```

Code Fragment 4.1: A recursive implementation of the factorial function.

This function does not use any explicit loops. Repetition is provided by the repeated recursive invocations of the function. There is no circularity in this definition, because each time the function is invoked, its argument is smaller by one, and when a base case is reached, no further recursive calls are made.

We illustrate the execution of a recursive function using a ***recursion trace***. Each entry of the trace corresponds to a recursive call. Each new recursive function call is indicated by a downward arrow to a new invocation. When the function returns, an arrow showing this return is drawn and the return value may be indicated alongside this arrow. An example of such a trace for the factorial function is shown in Figure 4.1.

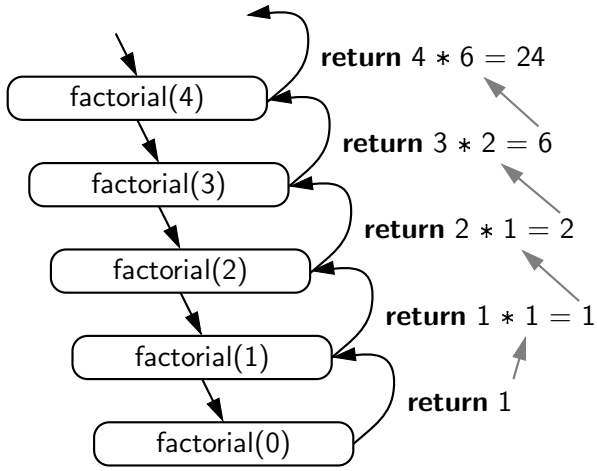


Figure 4.1: A recursion trace for the call factorial(5).

A recursion trace closely mirrors the programming language's execution of the recursion. In Python, each time a function (recursive or otherwise) is called, a structure known as an ***activation record*** or ***frame*** is created to store information about the progress of that invocation of the function. This activation record includes a namespace for storing the function call's parameters and local variables (see Section 1.10 for a discussion of namespaces), and information about which command in the body of the function is currently executing.

When the execution of a function leads to a nested function call, the execution of the former call is suspended and its activation record stores the place in the source code at which the flow of control should continue upon return of the nested call. This process is used both in the standard case of one function calling a different function, or in the recursive case in which a function invokes itself. The key point is that there is a different activation record for each active call.

4.1.2 Drawing an English Ruler

In the case of computing a factorial, there is no compelling reason for preferring recursion over a direct iteration with a loop. As a more complex example of the use of recursion, consider how to draw the markings of a typical English ruler. For each inch, we place a tick with a numeric label. We denote the length of the tick designating a whole inch as the ***major tick length***. Between the marks for whole inches, the ruler contains a series of ***minor ticks***, placed at intervals of $1/2$ inch, $1/4$ inch, and so on. As the size of the interval decreases by half, the tick length decreases by one. Figure 4.2 demonstrates several such rulers with varying major tick lengths (although not drawn to scale).

```
---- 0
-
--
-
---
-
--
-
---- 1
-
--
-
---
-
--
-
---- 2
(a)
```

```
----- 0
-
--
-
---
-
--
-
----
-
--
-
---
-
--
-
----- 1
(b)
```

```
--- 0
-
--
-
--- 1
-
--
-
--- 2
-
--
-
--- 3
(c)
```

Figure 4.2: Three sample outputs of an English ruler drawing: (a) a 2-inch ruler with major tick length 4; (b) a 1-inch ruler with major tick length 5; (c) a 3-inch ruler with major tick length 3.

A Recursive Approach to Ruler Drawing

The English ruler pattern is a simple example of a ***fractal***, that is, a shape that has a self-recursive structure at various levels of magnification. Consider the rule with major tick length 5 shown in Figure 4.2(b). Ignoring the lines containing 0 and 1, let us consider how to draw the sequence of ticks lying between these lines. The central tick (at 1/2 inch) has length 4. Observe that the two patterns of ticks above and below this central tick are identical, and each has a central tick of length 3.

In general, an interval with a central tick length $L \geq 1$ is composed of:

- An interval with a central tick length $L-1$
- A single tick of length L
- An interval with a central tick length $L-1$

Although it is possible to draw such a ruler using an iterative process (see Exercise P-4.25), the task is considerably easier to accomplish with recursion. Our implementation consists of three functions, as shown in Code Fragment 4.2. The main function, draw_ruler, manages the construction of the entire ruler. Its arguments specify the total number of inches in the ruler and the major tick length. The utility function, draw_line, draws a single tick with a specified number of dashes (and an optional string label, that is printed after the tick).

The interesting work is done by the recursive draw_interval function. This function draws the sequence of minor ticks within some interval, based upon the length of the interval's central tick. We rely on the intuition shown at the top of this page, and with a base case when $L = 0$ that draws nothing. For $L \geq 1$, the first and last steps are performed by recursively calling draw_interval$(L-1)$. The middle step is performed by calling the function draw_line(L).

```
1  def draw_line(tick_length, tick_label=''):
2    """Draw one line with given tick length (followed by optional label)."""
3    line = '-' * tick_length
4    if tick_label:
5      line += ' ' + tick_label
6    print(line)
7
8  def draw_interval(center_length):
9    """Draw tick interval based upon a central tick length."""
10   if center_length > 0:                    # stop when length drops to 0
11     draw_interval(center_length - 1)       # recursively draw top ticks
12     draw_line(center_length)               # draw center tick
13     draw_interval(center_length - 1)       # recursively draw bottom ticks
14
15 def draw_ruler(num_inches, major_length):
16   """Draw English ruler with given number of inches, major tick length."""
17   draw_line(major_length, '0')             # draw inch 0 line
18   for j in range(1, 1 + num_inches):
19     draw_interval(major_length - 1)        # draw interior ticks for inch
20     draw_line(major_length, str(j))        # draw inch j line and label
```

Code Fragment 4.2: A recursive implementation of a function that draws a ruler.

Illustrating Ruler Drawing Using a Recursion Trace

The execution of the recursive draw_interval function can be visualized using a recursion trace. The trace for draw_interval is more complicated than in the factorial example, however, because each instance makes two recursive calls. To illustrate this, we will show the recursion trace in a form that is reminiscent of an outline for a document. See Figure 4.3.

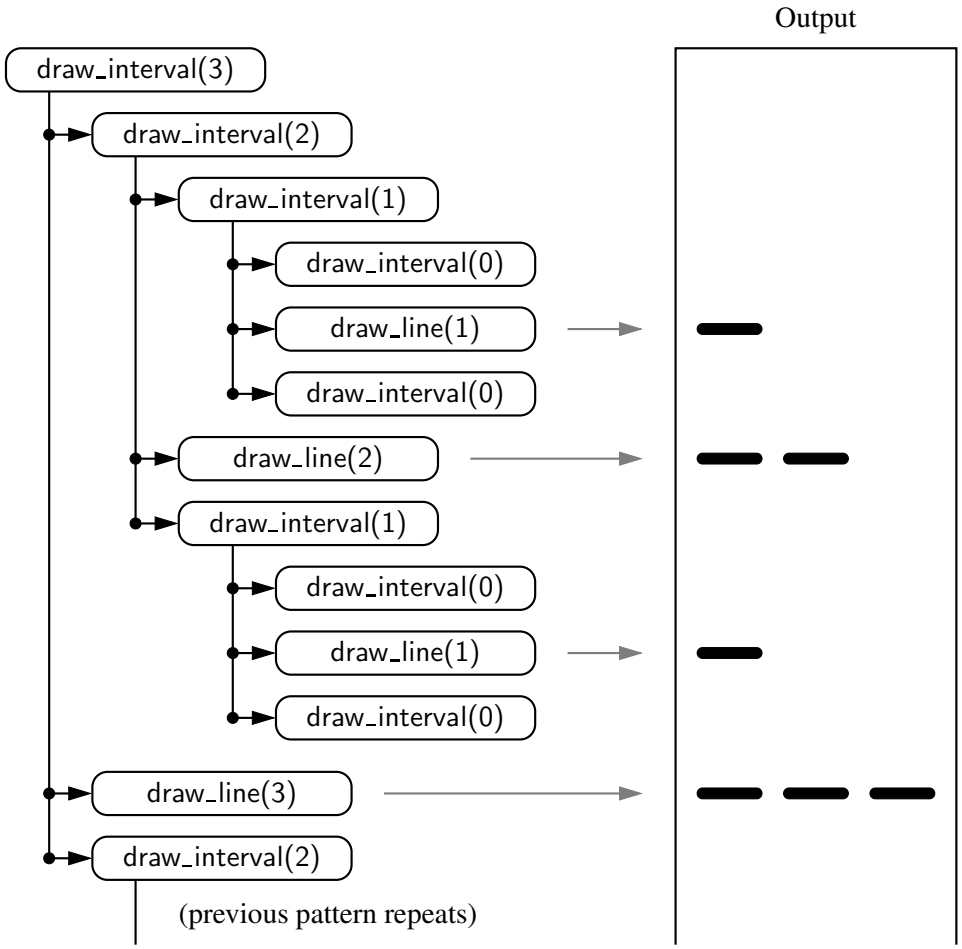


Figure 4.3: A partial recursion trace for the call draw_interval(3). The second pattern of calls for draw_interval(2) is not shown, but it is identical to the first.

4.1.3 Binary Search

In this section, we describe a classic recursive algorithm, ***binary search***, that is used to efficiently locate a target value within a sorted sequence of n elements. This is among the most important of computer algorithms, and it is the reason that we so often store data in sorted order (as in Figure 4.4).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37

Figure 4.4: Values stored in sorted order within an indexable sequence, such as a Python list. The numbers at top are the indices.

When the sequence is ***unsorted***, the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set. This is known as the ***sequential search*** algorithm. This algorithm runs in $O(n)$ time (i.e., linear time) since every element is inspected in the worst case.

When the sequence is ***sorted*** and ***indexable***, there is a much more efficient algorithm. (For intuition, think about how you would accomplish this task by hand!) For any index j, we know that all the values stored at indices $0,\ldots,j-1$ are less than or equal to the value at index j, and all the values stored at indices $j+1,\ldots,n-1$ are greater than or equal to that at index j. This observation allows us to quickly "home in" on a search target using a variant of the children's game "high-low." We call an element of the sequence a ***candidate*** if, at the current stage of the search, we cannot rule out that this item matches the target. The algorithm maintains two parameters, low and high, such that all the candidate entries have index at least low and at most high. Initially, $\mathsf{low} = 0$ and $\mathsf{high} = n-1$. We then compare the target value to the median candidate, that is, the item data[mid] with index

$$\mathsf{mid} = \lfloor (\mathsf{low} + \mathsf{high})/2 \rfloor .$$

We consider three cases:

- If the target equals data[mid], then we have found the item we are looking for, and the search terminates successfully.
- If target $<$ data[mid], then we recur on the first half of the sequence, that is, on the interval of indices from low to $\mathsf{mid}-1$.
- If target $>$ data[mid], then we recur on the second half of the sequence, that is, on the interval of indices from $\mathsf{mid}+1$ to high.

An unsuccessful search occurs if low $>$ high, as the interval $[\mathsf{low}, \mathsf{high}]$ is empty.

This algorithm is known as ***binary search***. We give a Python implementation in Code Fragment 4.3, and an illustration of the execution of the algorithm in Figure 4.5. Whereas sequential search runs in $O(n)$ time, the more efficient binary search runs in $O(\log n)$ time. This is a significant improvement, given that if n is one billion, $\log n$ is only 30. (We defer our formal analysis of binary search's running time to Proposition 4.2 in Section 4.2.)

```
1  def binary_search(data, target, low, high):
2    """Return True if target is found in indicated portion of a Python list.
3
4    The search only considers the portion from data[low] to data[high] inclusive.
5    """
6    if low > high:
7      return False                          # interval is empty; no match
8    else:
9      mid = (low + high) // 2
10     if target == data[mid]:               # found a match
11       return True
12     elif target < data[mid]:
13       # recur on the portion left of the middle
14       return binary_search(data, target, low, mid - 1)
15     else:
16       # recur on the portion right of the middle
17       return binary_search(data, target, mid + 1, high)
```

Code Fragment 4.3: An implementation of the binary search algorithm.

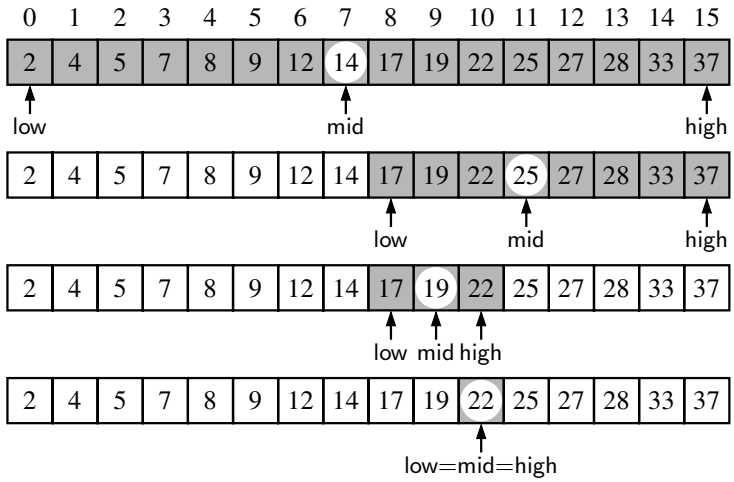


Figure 4.5: Example of a binary search for target value 22.

4.1.4 File Systems

Modern operating systems define file-system directories (which are also sometimes called "folders") in a recursive way. Namely, a file system consists of a top-level directory, and the contents of this directory consists of files and other directories, which in turn can contain files and other directories, and so on. The operating system allows directories to be nested arbitrarily deep (as long as there is enough space in memory), although there must necessarily be some base directories that contain only files, not further subdirectories. A representation of a portion of such a file system is given in Figure 4.6.

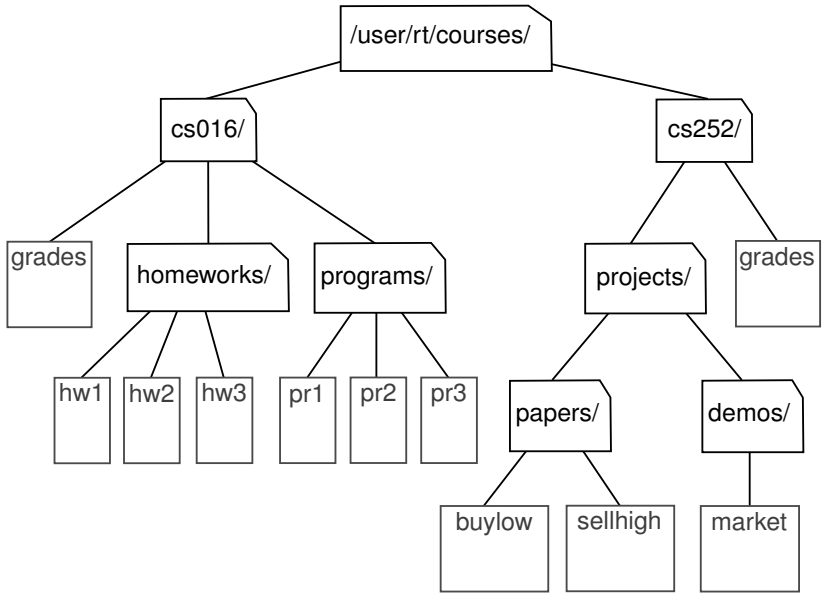


Figure 4.6: A portion of a file system demonstrating a nested organization.

Given the recursive nature of the file-system representation, it should not come as a surprise that many common behaviors of an operating system, such as copying a directory or deleting a directory, are implemented with recursive algorithms. In this section, we consider one such algorithm: computing the total disk usage for all files and directories nested within a particular directory.

For illustration, Figure 4.7 portrays the disk space being used by all entries in our sample file system. We differentiate between the *immediate* disk space used by each entry and the *cumulative* disk space used by that entry and all nested features. For example, the `cs016` directory uses only 2K of immediate space, but a total of 249K of cumulative space.

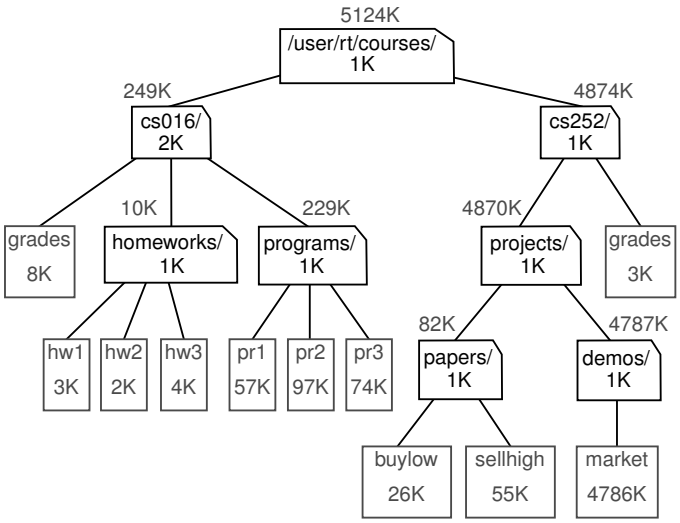


Figure 4.7: The same portion of a file system given in Figure 4.6, but with additional annotations to describe the amount of disk space that is used. Within the icon for each file or directory is the amount of space directly used by that artifact. Above the icon for each directory is an indication of the *cumulative* disk space used by that directory and all its (recursive) contents.

The cumulative disk space for an entry can be computed with a simple recursive algorithm. It is equal to the immediate disk space used by the entry plus the sum of the *cumulative* disk space usage of any entries that are stored directly within the entry. For example, the cumulative disk space for `cs016` is 249K because it uses 2K itself, 8K cumulatively in `grades`, 10K cumulatively in `homeworks`, and 229K cumulatively in `programs`. Pseudo-code for this algorithm is given in Code Fragment 4.4.

Algorithm DiskUsage(path):
Input: A string designating a path to a file-system entry
Output: The cumulative disk space used by that entry and any nested entries

```
total = size(path)                          {immediate disk space used by the entry}
if path represents a directory then
  for each child entry stored within directory path do
    total = total + DiskUsage(child)        {recursive call}
return total
```

Code Fragment 4.4: An algorithm for computing the cumulative disk space usage nested at a file-system entry. Function size returns the immediate disk space of an entry.

Python's os Module

To provide a Python implementation of a recursive algorithm for computing disk usage, we rely on Python's os module, which provides robust tools for interacting with the operating system during the execution of a program. This is an extensive library, but we will only need the following four functions:

- **os.path.getsize(path)**
  Return the immediate disk usage (measured in bytes) for the file or directory that is identified by the string path (e.g., `/user/rt/courses`).
- **os.path.isdir(path)**
  Return True if entry designated by string path is a directory; False otherwise.
- **os.listdir(path)**
  Return a list of strings that are the names of all entries within a directory designated by string path. In our sample file system, if the parameter is `/user/rt/courses`, this returns the list `['cs016', 'cs252']`.
- **os.path.join(path, filename)**
  Compose the path string and filename string using an appropriate operating system separator between the two (e.g., the / character for a Unix/Linux system, and the \ character for Windows). Return the string that represents the full path to the file.

Python Implementation

With use of the os module, we now convert the algorithm from Code Fragment 4.4 into the Python implementation of Code Fragment 4.5.

```
1  import os
2
3  def disk_usage(path):
4    """Return the number of bytes used by a file/folder and any descendents."""
5    total = os.path.getsize(path)                # account for direct usage
6    if os.path.isdir(path):                      # if this is a directory,
7      for filename in os.listdir(path):          # then for each child:
8        childpath = os.path.join(path, filename) # compose full path to child
9        total += disk_usage(childpath)           # add child's usage to total
10
11   print ('{0:<7}'.format(total), path)         # descriptive output (optional)
12   return total                                 # return the grand total
```

Code Fragment 4.5: A recursive function for reporting disk usage of a file system.

Recursion Trace

To produce a different form of a recursion trace, we have included an extraneous print statement within our Python implementation (line 11 of Code Fragment 4.5). The precise format of that output intentionally mirrors output that is produced by a classic Unix/Linux utility named du (for "disk usage"). It reports the amount of disk space used by a directory and all contents nested within, and can produce a verbose report, as given in Figure 4.8.

Our implementation of the disk_usage function produces an identical result, when executed on the sample file system portrayed in Figure 4.7. During the execution of the algorithm, exactly one recursive call is made for each entry in the portion of the file system that is considered. Because the print statement is made just before returning from a recursive call, the output shown in Figure 4.8 reflects the order in which the recursive calls are completed. In particular, we begin and end a recursive call for each entry that is nested below another entry, computing the nested cumulative disk space before we can compute and report the cumulative disk space for the containing entry. For example, we do not know the cumulative total for entry /user/rt/courses/cs016 until after the recursive calls regarding contained entries grades, homeworks, and programs complete.

```
8       /user/rt/courses/cs016/grades
3       /user/rt/courses/cs016/homeworks/hw1
2       /user/rt/courses/cs016/homeworks/hw2
4       /user/rt/courses/cs016/homeworks/hw3
10      /user/rt/courses/cs016/homeworks
57      /user/rt/courses/cs016/programs/pr1
97      /user/rt/courses/cs016/programs/pr2
74      /user/rt/courses/cs016/programs/pr3
229     /user/rt/courses/cs016/programs
249     /user/rt/courses/cs016
26      /user/rt/courses/cs252/projects/papers/buylow
55      /user/rt/courses/cs252/projects/papers/sellhigh
82      /user/rt/courses/cs252/projects/papers
4786    /user/rt/courses/cs252/projects/demos/market
4787    /user/rt/courses/cs252/projects/demos
4870    /user/rt/courses/cs252/projects
3       /user/rt/courses/cs252/grades
4874    /user/rt/courses/cs252
5124    /user/rt/courses/
```

Figure 4.8: A report of the disk usage for the file system shown in Figure 4.7, as generated by the Unix/Linux utility du (with command-line options -ak), or equivalently by our disk_usage function from Code Fragment 4.5.